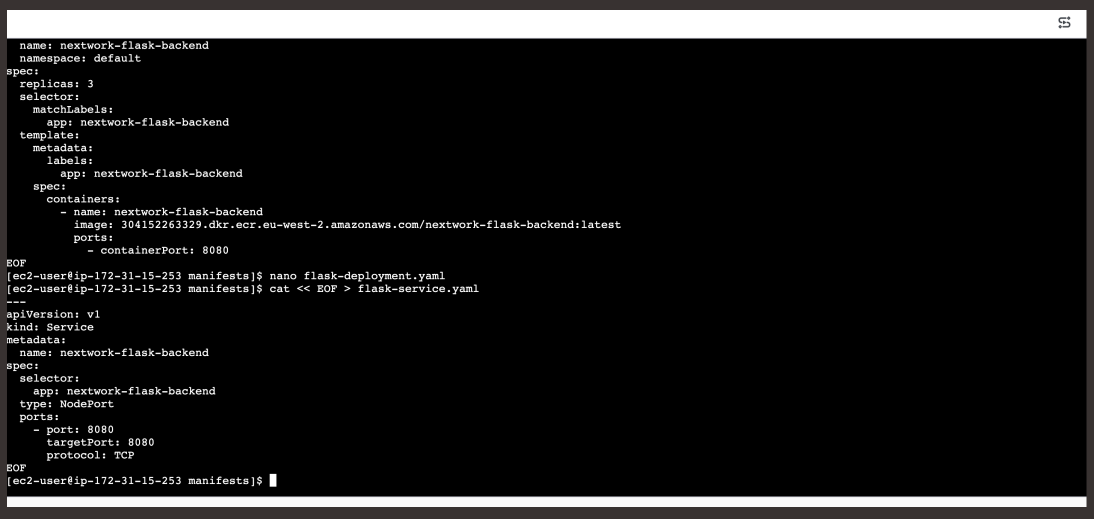
Create Kubernetes Manifests

JJ jjoseph1608@outlook.com



```
  name: nextwork-flask-backend
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nextwork-flask-backend
  template:
    metadata:
      labels:
        app: nextwork-flask-backend
    spec:
      containers:
        - name: nextwork-flask-backend
          image: 304152263329.dkr.ecr.eu-west-2.amazonaws.com/nextwork-flask-backend:latest
          ports:
            - containerPort: 8080
EOF
[ec2-user@ip-172-31-15-253 manifests]$ nano flask-deployment.yaml
[ec2-user@ip-172-31-15-253 manifests]$ cat << EOF > flask-service.yaml
---
apiVersion: v1
kind: Service
metadata:
  name: nextwork-flask-backend
spec:
  selector:
    app: nextwork-flask-backend
  type: NodePort
  ports:
    - port: 8080
      targetPort: 8080
      protocol: TCP
EOF
[ec2-user@ip-172-31-15-253 manifests]$
```

Introducing Today's Project!

In this project, I will deploy my backend application to my EKS cluster using Kubernetes Deployment and Service manifests. I'm doing this project to learn how to tell Kubernetes how to run my application and how to expose it so users can access it over the internet

Tools and concepts

I used Amazon EKS to create my Kubernetes cluster, eksctl to build it from the command line, Git to clone my backend code, Docker to containerize my application, and Amazon ECR to store my container images. Once everything was built, I used kubectl to deploy my Deployment and Service manifests to my EKS cluster. Key concepts include using manifests to declare what you want Kubernetes to do. A Deployment manifest tells Kubernetes which image to pull, how many replicas to run, and what ports to use. A Service manifest exposes those pods to the outside world and routes traffic to them. Pods are the smallest unit Kubernetes deploys, and Kubernetes uses labels and selectors to match Services to the Pods they route traffic to. By specifying replicas, Kubernetes automatically maintains that desired state and restarts failed pods

Project reflection

I chose to do this project today because I wanted to see my backend actually running in Kubernetes after building and containerizing it. Deploying it to a real cluster showed me how everything connects. Something that would make learning with NextWork better is building projects that evolve. Instead of isolated exercises, I'd like to take something I built and improve it in the next project. That would feel more like real engineering work. Also, more troubleshooting challenges where you figure things out yourself instead of being told the answer would help learning stick better

JJ
jjoseph1608@outlook....
NextWork Student
nextwork.org

This project took me around 60 minutes to complete. The setup of the EKS cluster went faster the second time since I'd done it before. The longest part was creating and debugging the Deployment and Service manifests because I had to understand what each field did before I could write them correctly. Getting the manifests right so Kubernetes could actually pull my image from ECR and expose the service took some trial and error. My favorite part was seeing my backend actually running in Kubernetes. Once I deployed the manifests with kubectl apply, I could see the pods spinning up and the Service getting exposed. It felt like everything from the previous projects finally came together. I built the image in Docker, pushed it to ECR, and now it was actually running in a real Kubernetes cluster. That's the part that made it click for me

Project Set Up

Kubernetes cluster

To set up today's project, I launched a Kubernetes cluster. Steps I took to do this included launching an EC2 instance and connecting to it using EC2 Instance Connect. Then I installed eksctl on the instance and ran aws configure to set up my AWS credentials with my access key and secret key. After that I ran the eksctl create cluster command with parameters for the cluster name, node type, number of nodes, Kubernetes version, and region. Once CloudFormation finished building the cluster, I opened the EKS console and set up an IAM access entry for myself so I could interact with the cluster and deploy applications to it

Backend code

I retrieved the backend that I plan to deploy by running git clone followed by the GitHub repository URL. This downloaded the entire backend application to my instance so I could reference it when creating my Kubernetes manifests. Backend is the server side code that runs the actual business logic, handles requests, and serves data to users. In this case my backend is a Flask application that connects to the Hacker News Search API, fetches content based on a topic, and returns it as JSON

Container image

Once I cloned the backend code, I built a container image because I needed the containerized version to push to ECR and deploy to my Kubernetes cluster. I used the docker build command with the tag nextwork-flask-backend to create the image from the Dockerfile in the cloned repository

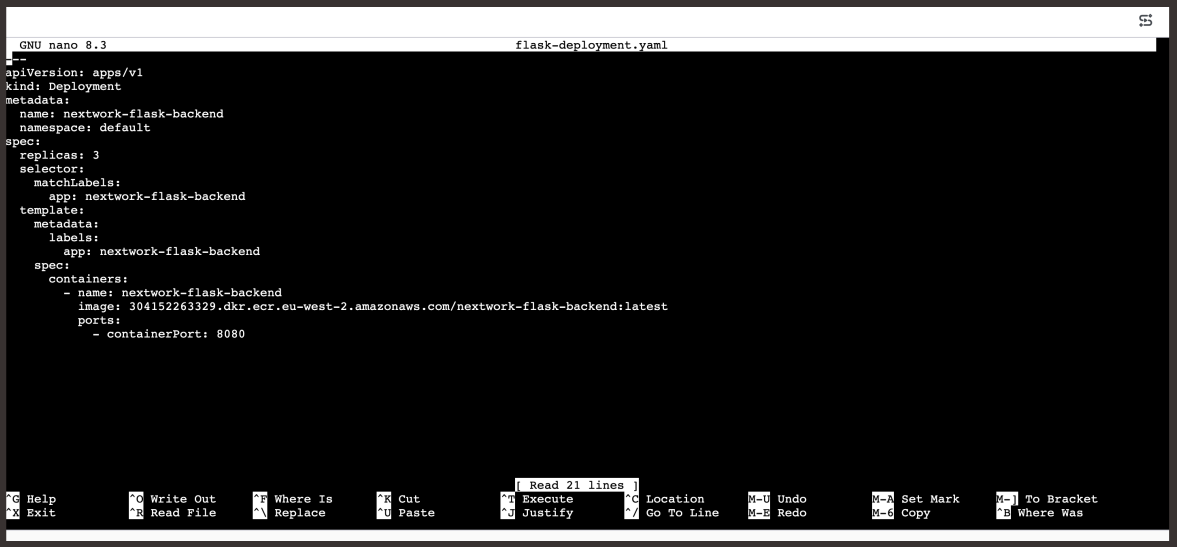
I also pushed the container image to a container registry because my Kubernetes cluster needs to access the image from ECR in order to pull it and deploy it. To push the image to ECR, I first created an ECR repository in the AWS console, then ran aws ecr get-login-password to authenticate Docker with ECR. Next I tagged my image with the ECR repository URL using docker tag, and finally pushed it to ECR using docker push. Once the image was in ECR, my EKS cluster could pull it whenever I deployed the application

Manifest files

Kubernetes manifests are configuration files that tell Kubernetes what resources you want to create and how to configure them. They're written in YAML format and describe things like deployments, services, and other Kubernetes objects. Manifests are helpful because they let you declare what you want Kubernetes to do. Instead of running commands one by one, you write a manifest once that says "I want 3 replicas of my backend running, using this image, on port 5000". Kubernetes reads that manifest and makes it happen. If a pod crashes, Kubernetes automatically spins up a new one to maintain that desired state. You don't have to manually manage it

A Kubernetes deployment manages the running instances of your application. It ensures the desired number of replicas are always running, handles rolling updates, and can automatically restart pods if they crash or fail. The container image URL in my Deployment manifest tells Kubernetes where to pull the image from. In my case, it points to my Amazon ECR repository so Kubernetes knows to pull my backend image from there when it deploys the application

JJ
jjoseph1608@outlook....
NextWork Student



Service Manifest

A Kubernetes Service exposes your pods to both inside and outside the cluster. It creates a stable IP address and DNS name so users and other services can access your application without needing to know which specific pod is running. You need a Service manifest to give your Deployment a way for traffic to reach it. When you deploy your backend, it runs in pods, but pods are temporary and can be created or destroyed. A Service sits in front of those pods and routes traffic to them. If a pod crashes and a new one starts, the Service still works because it's not tied to a specific pod. It's tied to all pods with matching labels

My Service manifest sets up a NodePort service that exposes my backend application on port 8080. It uses a label selector to find all pods running my backend that have the label app: nextwork-flask-backend. When users connect to port 8080 on any node in my cluster, traffic gets routed to port 8080 inside the containers running my Flask backend. The NodePort type means Kubernetes opens a port on each node and listens for incoming traffic on that port.

```
  name: nextwork-flask-backend
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nextwork-flask-backend
  template:
    metadata:
      labels:
        app: nextwork-flask-backend
    spec:
      containers:
        - name: nextwork-flask-backend
          image: 304152263329.dkr.ecr.eu-west-2.amazonaws.com/nextwork-flask-backend:latest
          ports:
            - containerPort: 8080
EOF
[ec2-user@ip-172-31-15-253 manifests]$ nano flask-deployment.yaml
[ec2-user@ip-172-31-15-253 manifests]$ cat << EOF > flask-service.yaml
---
apiVersion: v1
kind: Service
metadata:
  name: nextwork-flask-backend
spec:
  selector:
    app: nextwork-flask-backend
  type: NodePort
  ports:
    - port: 8080
      targetPort: 8080
      protocol: TCP
EOF
[ec2-user@ip-172-31-15-253 manifests]$
```

Deployment Manifest

Annotating the Deployment manifest helped me understand how Kubernetes actually deploys my backend because it forced me to look at each line and explain what it does. Instead of just copying a manifest and running it, I had to understand what the apiVersion means, what the selector does, why the replicas matter, and how the container spec tells Kubernetes which image to pull. Going through it piece by piece made me realize that a Deployment is really just instructions in YAML format. There's no magic. I'm telling Kubernetes exactly what I want, and it follows those instructions

A notable line in the Deployment manifest is the number of replicas, which means how many copies of your application you want running. If you specify 3 replicas, Kubernetes will ensure 3 copies of your backend are always running. Pods are relevant to this because a Pod is the smallest unit Kubernetes actually deploys. It's a wrapper around your Docker container. When you specify 3 replicas in your Deployment, Kubernetes creates 3 Pods, and each Pod runs one instance of your backend container. If a Pod crashes, Kubernetes sees that you now have only 2 replicas running when you want 3, so it automatically creates a new Pod to bring you back to the desired state. That's what makes Kubernetes powerful. You declare the desired state, and Kubernetes maintains it

One part of the Deployment manifest I still want to know more about is how Kubernetes authenticates with Amazon ECR to pull the container image because the manifest just has the image URL but it doesn't show any credentials. I'm wondering if there's some configuration I'm missing or if EKS handles that automatically through IAM roles. It seems like a security gap to put the ECR URL in a manifest without any way to authenticate, so I'd like to understand how that actually works behind the scenes

The place to learn & showcase your skills

Check out nextwork.org for more projects